# Report of Industrial Training

on

Developing **self-adjusting** robotic welding systems for manufacturing of **augers**.


*Submitted By*:

**Ghanashyam R K P**

Reg. No. 180905042

Section A

Roll. No. 8

Email: ghanashyamrkp@gmail.com

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
*(A constituent unit of MAHE, Manipal)*

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING**

*Under the Guidance of:*

**Dr. Matt Khoshdarregi**

Assistant Professor

Intelligent Digital Manufacturing Lab,

University of Manitoba, Winnipeg, Canada.

University of Manitoba


**July - September 2021**

# Certificate of Completion

**MITACS Globalink**
**CERTIFICATE OF COMPLETION**

***Ghanashyam Ramia Kasthurirangan Pandurengan***

This certifies that Ghanashyam Ramia Kasthurirangan Pandurengan has successfully completed a Mitacs Globalink Research Internship effective October 2021.

**Project Title:** **Intelligent Vision-Based Inspection Techniques for Robotic Manufacturing Systems**
**Host University:** **University of Manitoba – Winnipeg**
**Host Professor:** **Matt Khoshdarregi**

Globalink Research Internships are 12 weeks in duration and interns are required to work 40 hours per week.

**October 10th, 2021**

**Gail Bowkett**
Vice President of Programs

Mitacs

# Acknowledgement

I would like to express my sincere thanks to Dr. Matt Khoshdarregi, for his valuable guidance and support in the program and for providing a vision for the project and his unfailing cooperation to assist me in my work. I benefited greatly from the weekly meetings, which helped me in making crucial decisions and changing my viewpoints and direction of thought regarding the project's development across the internship duration.

I am thankful to Mr. Ali Maghami and Mr. Bhavin Dharia (Mentors), for their valuable suggestions over the technicalities of the project. Being senior students of that college and having gone through the same path previously once in their projects, they were able to perfectly guide and infer my thoughts during the weekly online meetings and provide suggestions on the same.

I would also like to thank my teammate - fellow intern on the same project, Mr. Abhijay Kemkar, BITS Pilani, Rajasthan, with whom I comfortably discussed topics and had separate brainstorming sessions in order to complete the project.

Last but not least, my sincere thanks to MITACS, for providing a wonderful opportunity to intern at Canada, through the Globalink Research Internship Programme. I would like to express my gratitude to them for considering me eligible for this programme and for providing me resources for the smooth completion of the internship.

# Abstract

Welding of augers often requires continuous human supervision, from the start of the welding process till the end of it. Generally the manipulator with the welding bit is part-human controlled in a sense that it has to be stopped in the middle to appropriately turn the auger and do it repeatedly until the welding is done.

One more problem in current welding process is that the weld-path (path the manipulator has to follow to weld the helical part and the cylindrical part) is not determined beforehand and the manipulator is usually directed by a human to show where it has to weld, and how.

To tackle the above problems, me and my teammate* together have developed a method in which there is very minimal human intervention, and the welding process of the auger, starting from the path detection to welding of the auger is done autonomously.

It is a generalised algorithm which will work for any auger irrespective of its size. Right now, the algorithm was developed for the phase of feasibility study of the original project, which then will be implemented with specific changes in a real environment. All developed algorithms work in a **simulation environment** without bugs.

* We both divided our work and did our parts completely individually. None of the work was repeated.

# Contents

# Details about the organisation

## MITACS

Mitacs is a nonprofit national research organization that, in partnerships with Canadian academia, private industry and government, operates research and training programs in fields related to industrial and social innovation.

Globalink Research Internship is a competitive initiative for international undergraduates from the following countries and regions: Australia, Brazil, China, Colombia, France, Germany, Hong Kong, **India**, Mexico, Pakistan, Taiwan, Tunisia, Ukraine, United Kingdom and the United States. From May to October of each year, top-ranked applicants participate in a 12-week research internship under the supervision of Canadian university faculty members in a variety of academic disciplines, from science, engineering, and mathematics to the humanities and social sciences.

## University of Manitoba

The University of Manitoba (U of M, UManitoba, or UM) is a non-denominational, public research university in the province of Manitoba, Canada, Founded in 1877, it is the first university of western Canada.

The university maintains a reputation as a top research-intensive post-secondary educational institution, conducting more research annually than any other university in the region; its competitive academic and research programs have also consistently ranked among the top in the Canadian Prairies. Research at the University of Manitoba has accordingly produced various world-renowned contributions, including the creation of canola oil in the 1970s. Likewise, U of M alumni include Nobel Prize recipients, Academy Award winners, Order of Merit recipients, and Olympic medalists, among many others. As of 2019, there have been 99 Rhodes Scholarship recipients from the U of M, more than that of any other university in western Canada. Moreover, the university has produced countless government figures, including provincial premiers, Supreme Court justices, and Members of Parliament (MPs).

## Intelligent Digital Manufacturing Lab (IDML)

Research at IDML

The long-term vision for our research is to develop next-generation Intelligent Manufacturing Systems (IMS) to realize the seamless end-to-end integration of the manufacturing chain, from design and process planning to production, process monitoring, and inspection. In addition to lowering costs, decreasing lead times, and improving product quality, intelligent systems provide significant flexibility and allow for the manufacturing of individually-customized products on mass production lines.

# Introduction

## 1.1 Motivation

The project I was assigned as part of the MITACS Globalink research internship was related to Software Development for Robots, specifically developing code that would fully automate the welding process done by a Six Degree of Freedom Robotic Manipulator.

So, a question that naturally arises is, what are the prerequisites for the project and how did I manage to gather it in such a short period of time?

To answer this, I would have to start from the start of the year 2020. I worked as an AI researcher in one of the best student project teams of MIT - Mars Rover Manipal. So, during the task-phase there, our team was working on automating a Seven Degree of Freedom Robotic Manipulator and I was one of the team members responsible for the development of software to automate it.

To achieve it, I had learnt Robot Operating System, which is a middleware between the hardware of the robot and the programs that we write, which then helps to perform automation tasks on it. Along with ROS, I also had to learn specific tools like Moveit - which is a motion planning framework for robotic arms, and a simulation software called gazebo.

One interesting thing to note here is that every small detail had to be done from scratch - from creating specific packages for the manipulator till performing an action like “Pick and Place”, using the manipulator, as the manipulator we developed for the project at MRM was not an industrial manipulator, but instead my mechanical colleagues had developed it from scratch for the project.

This struggle of doing everything from scratch without beginning gave me enough Motivation and Confidence to apply for the MITACS Globalink research Project, which was specifically related to automating a robot.

## 1.2 Project Explanation and Work Taken

This project as explained in the abstract above is to fully automate a 6-DoF industrial robotic manipulator for welding of augers.

Fig 1.1 Auger

The above picture of auger has two separate components - Cylindrical and Helical Components, and the broad - goal of the project is to weld the intersection part of them using an industrial robotic manipulator automatically without any manual intervention, except for placing them in their positions.

So, the first part of the project is to identify the intersection of the helical and cylindrical components, so that the manipulator can follow that path and weld the helical part to the cylindrical part.

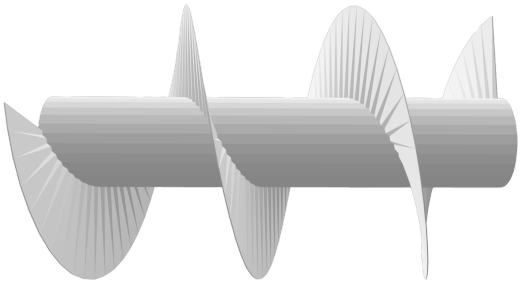


Fig 1.2 CAD Model of Auger

So the CAD model of auger as shown in Fig 1.2 was developed and then computer vision was used to extract the seam of the auger. This process was done by the other intern. I also had a contribution on extracting the intersection path, where I devised an algorithm for rotating the auger. More on that later in the report.

As the first part of the project was over, I was provided with the weld path - Fig 1.3, of the auger. My part of the internship was to devise an algorithm that would weld the helical component with the cylindrical component.

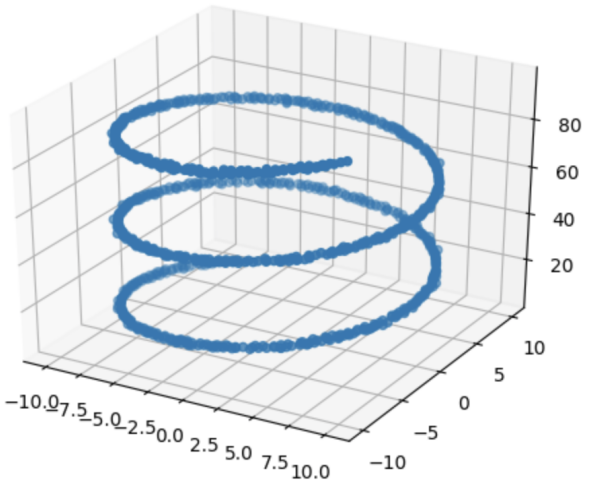



Fig. 1.3 Intersection / Welding path

My Goals :

a) Rotating the auger
b) Simultaneously moving the weld bit (attached with the end-effector of the robotic arm), in order to weld the intersection.

### 1.3 Training Details

This section includes details about the frameworks and concepts learnt in-order to achieve the internship goals.

Frameworks Learnt and it's details:

**Robot Operating System**

Robot Operating System is an open-source robotics middleware suite. Although ROS is not an operating system but a collection of software frameworks for robot software development, it provides services designed for a heterogeneous computer cluster such as hardware abstraction, low-level device control, implementation of commonly used functionality, message-passing between processes, and package management. Running sets of ROS-based processes are represented in a graph architecture where processing takes place in nodes that may receive, post and multiplex sensor data, control, state, planning, actuator, and other messages. Despite the importance of reactivity and low latency in robot control, ROS itself is *not* a real-time OS (RTOS). It is possible, however, to integrate ROS with real-time code. The lack of support for real-time systems has been addressed in the creation of ROS 2, a major revision of the ROS API which will take advantage of modern libraries and technologies for core ROS functionality and add support for real-time code and embedded hardware.

Software in the ROS Ecosystem can be separated into three groups:

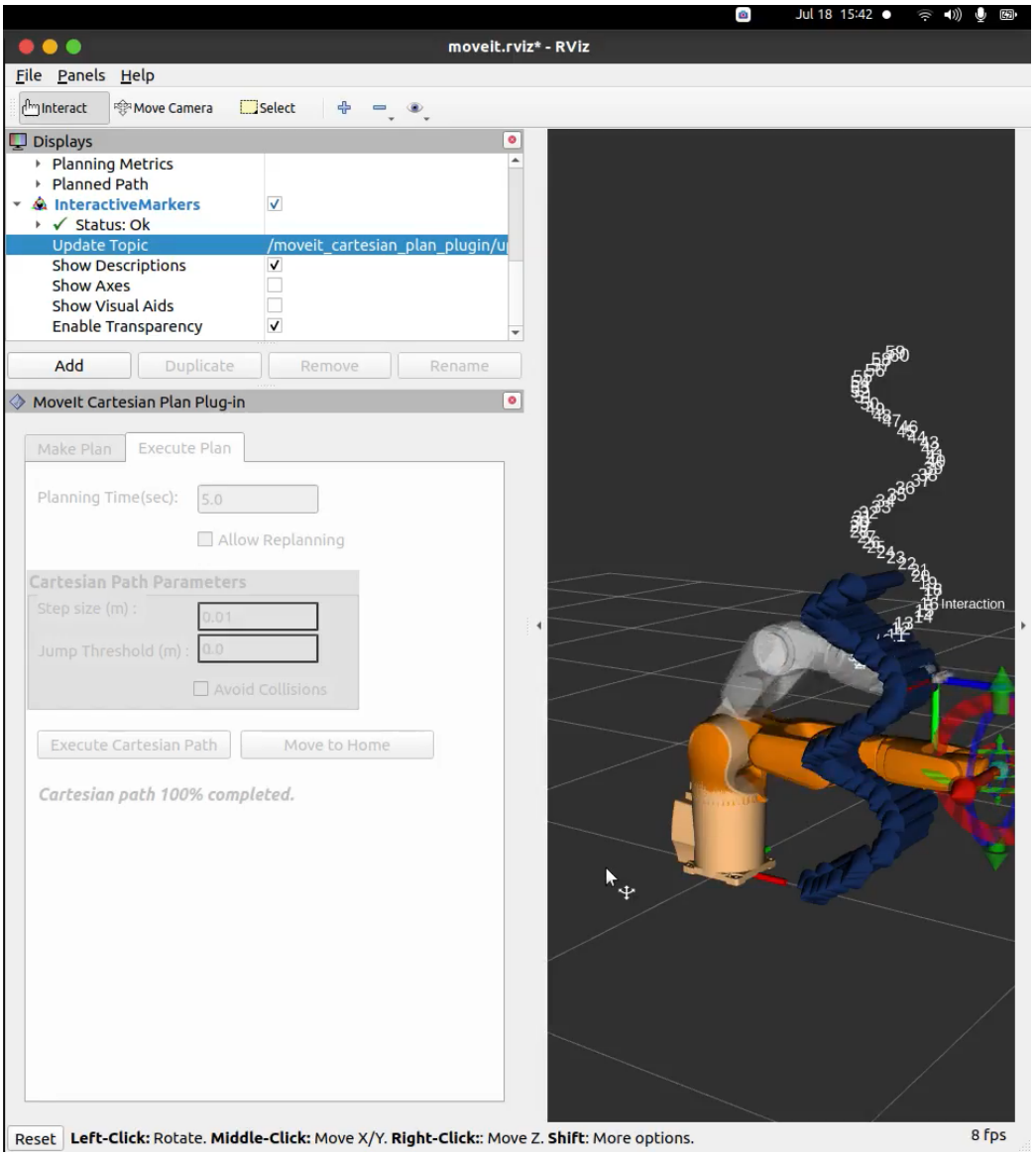
- language-and platform-independent tools used for building and distributing ROS-based software;
- ROS client library implementations such as roscpp,rospy, and roslisp;
- packages containing application-related code which uses one or more ROS client libraries.

Both the language-independent tools and the main client libraries (C++, Python, and Lisp) are released under the terms of the BSD license, and as such are open-source software and free for both commercial and research use. The majority of other packages are licensed under a variety of open-source licenses. These other packages implement commonly used functionality and applications such as hardware drivers, robot models, data types, planning, perception, simultaneous localization and mapping, simulation tools, and other algorithms.

**Moveit Motion Planning Framework**

This framework is mainly used for motion planning for a robotic manipulator. In detail, it's features are:

(A) Motion planning - It generates high degree of freedom trajectories through cluttered environments and avoid local minimums
(B) Manipulation - For analysing and interacting with the environment with grasp generation.
(C) Inverse Kinematics - Solve for joint positions for a given pose, even in over-actuated arms
(D) Control - Execute time-parameterized joint trajectories to low level hardware controllers through interfaces.
(E) Collision Checking - Avoid obstacles using geometric primitives, meshes, or point cloud data.

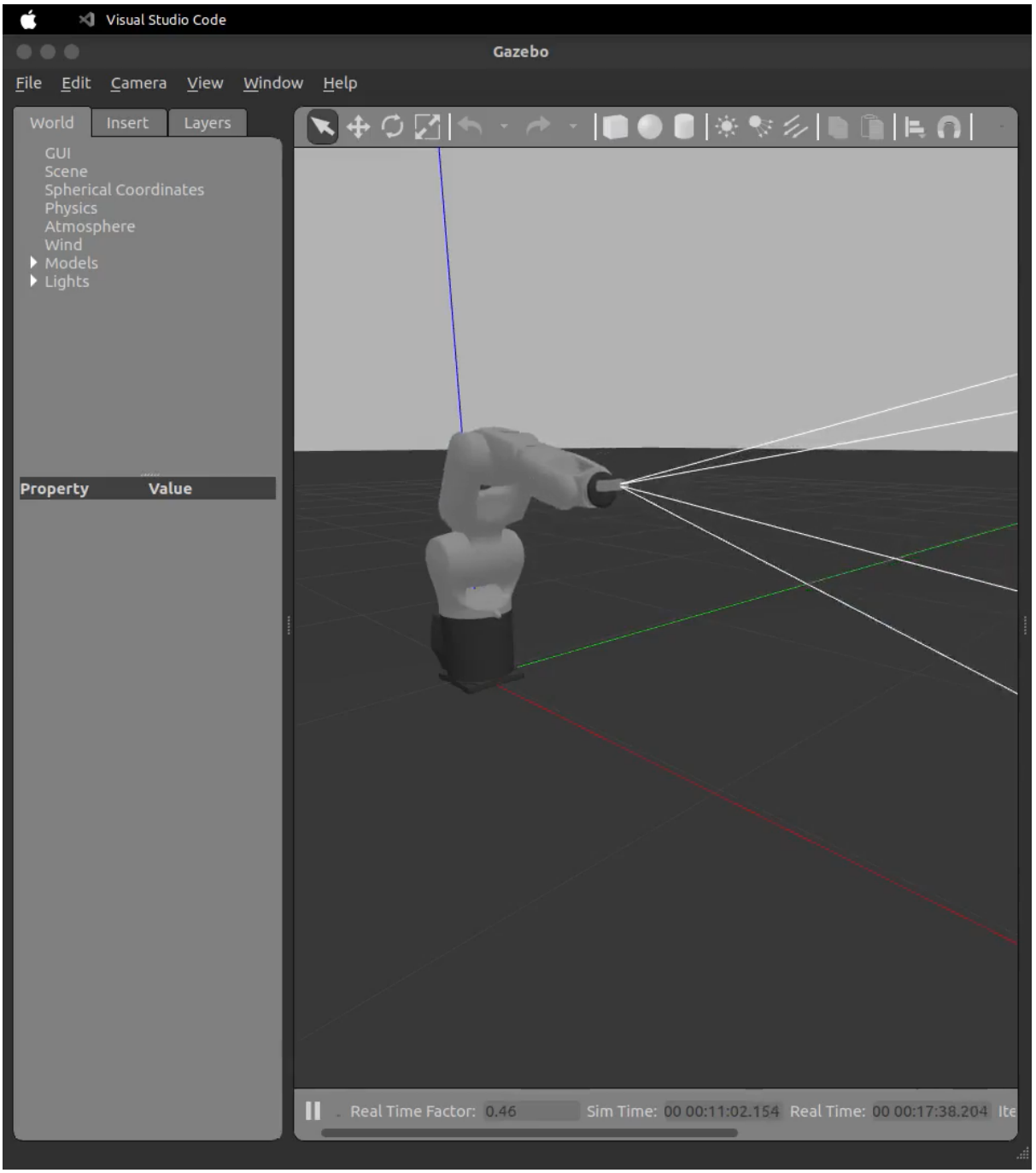



Key Assisting interfaces are:

(a) Gazebo: Can speed up development and testing cycles by leveraging a full physics-based simulator with MoveIt. We can combine Gazebo, ROS Control, and MoveIt for a powerful robotics development platform.
(b) 3D Interactive Visualizer: Out-of-the box visual demonstrations in Rviz allow new users experimentation with various planning algorithms around obstacles. Execution on physical hardware is then just a click away.
(c) Moveit Setup Assistant: Any robot can be quickly set up to plan and execute motion planning. It is a step-by-step configuration wizard, which uses popular pre-configured setups. Also includes configuration of Gazebo and ROS Control.
(d) Task Constructor: A flexible and transparent way to define and plan actions that consist of multiple interdependent subtasks.
(e) Grasp Generation: Libraries for geometric and machine learning-based grasp generation for use with the MoveIt pick and place pipeline.

**Gazebo:**

Robot simulation is an essential tool in every roboticist's toolbox. A well-designed simulator makes it possible to rapidly test algorithms, design robots, perform regression testing, and train AI systems using realistic scenarios. Gazebo offers the ability to accurately and efficiently simulate populations of robots in complex indoor and outdoor environments. At your fingertips is a robust physics engine, high-quality graphics, and convenient programmatic and graphical interfaces. Best of all, Gazebo is free with a vibrant community.




**Programming Languages Used:**

Since in Robot Operating System, it is language agnostic, one can use C++, python and lisp simultaneously for coding in the same project.

So I used, both C++ and Python for my project.

I learnt how to create Libraries in C++ and Python to use them in my code. They were useful while I wanted to execute complex actions like rotating a point cloud. The function I wrote was able to use it everywhere in the code and that helped me rotate a point cloud.

Other Libraries used:

(a) Open3D: Open3D is an open-source library that supports rapid development of software that deals with 3D data. The Open3D frontend exposes a set of carefully selected data structures and algorithms in both C++ and Python. The backend is highly optimized and is set up for parallelization. Open3D was developed from a clean slate with a small and carefully considered set of dependencies. It can be set up on different platforms and compiled from source with minimal effort. The code is clean, consistently styled, and maintained via a clear code review mechanism. Open3D has been used in a number of published research projects and is actively deployed in the cloud. We welcome contributions from the open-source community.

I used Open3D for Rotation of the point cloud.

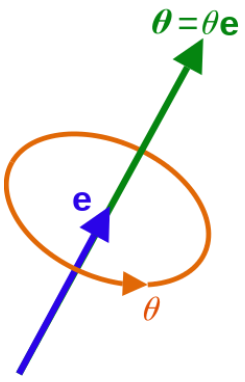
(b) Point Cloud Library: The Point Cloud Library (PCL) is a standalone, large scale, open project for 2D/3D image and point cloud processing. PCL is released under the terms of the BSD license, and thus free for commercial and research use.

**Mathematics Used:**

Used basic trigonometry and learnt and used advanced concepts like axis angle formula to devise an algorithm that would simultaneously rotate the auger by the calculated angles using that formula.

**Axis-Angle Concept:**

In mathematics, the axis–angle representation of a rotation parameterizes a rotation in a three-dimensional Euclidean space by two quantities: a unit vector **e** indicating the direction of an axis of rotation, and an angle **θ** describing the magnitude of the rotation about the axis. Only two numbers, not three, are needed to define the direction of a unit vector e rooted at the origin because the magnitude of **e** is constrained. For example, the elevation and azimuth angles of **e** suffice to locate it in any particular Cartesian coordinate frame.



By Rodrigues' rotation formula, the angle and axis determine a transformation that rotates three-dimensional vectors. The rotation occurs in the sense prescribed by the right-hand rule. The rotation axis is sometimes called the Euler axis.

Let me explain what a rotation vector is:

The axis–angle representation is equivalent to the more concise rotation vector, also called the Euler vector. In this case, both the rotation axis and the angle are represented by a vector codirectional with the rotation axis whose length is the rotation angle θ,

$$\boldsymbol{\theta = \theta e}$$

Many rotation vectors correspond to the same rotation. In particular, a rotation vector of length θ + 2πM, for any integer M, encodes exactly the same rotation as a rotation vector of length θ. Thus, there are at least a countable infinity of rotation vectors corresponding to any rotation. Furthermore, all rotations by 2πM are the same as no rotation at all, so, for a given integer M, all rotation vectors of length 2πM, in all directions, constitute a two-parameter uncountable infinity of rotation vectors encoding the same rotation as the zero vector

Example:

Let's take the direction of gravity to be the negative z axis. Considering we are standing on the ground and turn left to rotate π/2 radians (or 90°) about the z axis. Viewing the axis-angle representation as an ordered pair, this would be

$$(\text{axis}, \text{angle}) = \left( \begin{bmatrix} e_x \\ e_y \\ e_z \end{bmatrix}, \theta \right) = \left( \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \frac{\pi}{2} \right)$$

So the rotation vector will be a vector with a magnitude of π/2 pointing in the Z direction,

$$\begin{bmatrix} 0 \\ 0 \\ \frac{\pi}{2} \end{bmatrix} .$$

**Quaternions:**

In mathematics, the quaternion number system extends the complex numbers. Quaternions were first described by Irish mathematician William Rowan Hamilton in 1843 and applied to mechanics in three-dimensional space. Hamilton defined a quaternion as the quotient of two directed lines in a three-dimensional space, or, equivalently, as the quotient of two vectors. Multiplication of quaternions is noncommutative.

Quaternions are generally represented in the form

$$a + b\,\mathbf{i} + c\,\mathbf{j} + d\,\mathbf{k}$$

where a, b, c, and d are real numbers; and i, j, and k are the basic quaternions.

Quaternions are used in pure mathematics, but also have practical uses in applied mathematics, particularly for calculations involving three-dimensional rotations, such as in three-dimensional computer graphics, computer vision, and crystallographic texture analysis. They can be used alongside other methods of rotation, such as Euler angles and rotation matrices, or as an alternative to them, depending on the application.

This Concept was used by ROS as an Alternate form to represent the orientation of a 3D object. There is a function in ROS which can also convert from quaternion to euler angles and vice-versa.

**Software used to code:**

**Google Colab:** It is a platform similar to a jupyter notebook and was mainly used to test the written algorithms chunk by chunk which were then converted to ROS nodes.

**Visual Studio Code:** This IDE helped in keeping all the ROS related packages in one chunk which helped in managing the packages effectively. The autocorrect, autocomplete and highlighting of code helped in writing long pieces of code effectively. It also supports compiling ROS packages and code auto-suggestions.

### 1.4 Terminologies used in ROS

I'm writing this separately here as these are some of the required terminologies without understanding which the report will not make sense further. So it is advised to read them carefully

Packages: They are the main unit for organizing software in ROS. A package may contain ROS runtime processes (nodes), a ROS dependent Library, datasets, configuration files, or anything else that is usefully organized together. They are the most atomic build item and release item in ROS.

In ROS, a computation graph is a peer-to-peer network of ROS processes that are processing data together. The basic Computation Graph concepts of ROS are *Nodes, Master, Parameter Server, messages, services, topics, and bags*, all of which provide data to the Graph in different ways.

**Nodes**: Nodes are processes that perform computation. ROS is designed to be modular at a fine-grained scale; a robot control system usually comprises many nodes. For example, one node controls a laser range-finder, one node controls the wheel motors, one node performs localization, one node performs path planning, one Node provides a graphical view of the system, and so on. A ROS node is written with the use of a ROS client library, such as roscpp or rospy.

**Master**: The ROS Master provides name registration and lookup to the rest of the Computation Graph. Without the Master, nodes would not be able to find each other, exchange messages, or invoke services.

**Parameter Server**: The Parameter Server allows data to be stored by key in a central location. It is currently part of the Master.

**Messages**: Nodes communicate with each other by passing messages. A message is simply a data structure, comprising typed fields. Standard primitive types (integer, floating point, boolean, etc.) are supported, as are arrays of primitive types. Messages can include arbitrarily nested structures and arrays (much like C structs).

**Topics**: Messages are routed via a transport system with publish / subscribe semantics. A node sends out a message by publishing it to a given topic. The topic is a name that is used to identify the content of the message. A node that is interested in a certain kind of data will subscribe to the appropriate topic. There may be multiple concurrent publishers and subscribers for a single topic, and a single node may publish and/or subscribe to multiple topics. In general, publishers and subscribers are not aware of each others' existence. The idea is to decouple the production of information from its consumption. Logically, one can think of a topic as a strongly typed message bus. Each bus has a name, and anyone can connect to the bus to send or receive messages as long as they are the right type.

**Services**: The publish / subscribe model is a very flexible communication paradigm, but its many-to-many, one-way transport is not appropriate for request / reply interactions, which are often required in a distributed system. Request / reply is done via services, which are defined by a pair of message structures: one for the request and one for the reply. A providing node offers a service under a name and a client uses the service by sending the request message and awaiting the reply. ROS client libraries generally present this interaction to the programmer as if it were a remote procedure call.

**Bags**: Bags are a format for saving and playing back ROS message data. Bags are an important mechanism for storing data, such as sensor data, that can be difficult to collect but is necessary for developing and testing algorithms.

The ROS Master acts as a nameservice in the ROS Computation Graph. It stores topics and services registration information for ROS nodes. Nodes communicate with the Master to report their registration information. As these nodes communicate with the Master, they can receive information about other registered nodes and make connections as appropriate. The Master will also make callbacks to these nodes when this registration information changes, which allows nodes to dynamically create connections as new nodes are run.

Nodes connect to other nodes directly; the Master only provides lookup information, much like a DNS server. Nodes that subscribe to a topic will request connections from nodes that publish that topic, and will establish that connection over an agreed upon connection protocol. The most common protocol used in a ROS is called TCPROS, which uses standard TCP/IP sockets.

This architecture allows for decoupled operation, where the names are the primary means by which larger and more complex systems can be built. Names have a very important role in ROS: nodes, topics, services, and parameters all have names.

Every ROS client library supports command-line remapping of names, which means a compiled program can be reconfigured at runtime to operate in a different Computation Graph topology.

# Detailed Explanation of the Project

### 2.1 Learning Phase

#### 2.1.1 Library creation in ROS using python and c++

### 2.2 Rotating a point cloud

I started my work when my fellow intern wanted to scan the auger for generation of the point cloud using his code. But as the auger is cylindrical, we needed to rotate the auger to 180 degrees in-order to generate the 3D point cloud of the auger.

I tried both ways (i.e) using C++ (with PCL) and Python (with Open3D) to rotate the point cloud.

At start I created my own custom library “rotate_pc.h” which would import all necessary libraries of PCL and also declares a function “rotate_pc” which will be defined in a separate file called “rotate_pc.cpp”. It’s done that way as the process of writing a library in c++, includes writing of a “.h” file also called a header file, which would include all the function declarations and its definition would be written in a separate file and compiled.

So using this library I created another node in the main package called “tester_package”,

```
class cloudHandler
{
public:
    cloudHandler()
    {
        pcl_sub = nh.subscribe("pcl_output", 10, &cloudHandler::cloudCB, this);
        pcl_pub = nh.advertise<sensor_msgs::PointCloud2>("pcl_rotated", 1);
    }

    void cloudCB(const boost::shared_ptr<const sensor_msgs::PointCloud2>& input)
    {
        pcl::PCLPointCloud2 pcl_pc2;
        pcl_conversions::toPCL(*input, pcl_pc2);

        pcl::PointCloud<pcl::PointXYZ>::Ptr source_cloud (new pcl::PointCloud<pcl::PointXYZ> ());
        pcl::fromPCLPointCloud2(pcl_pc2, *source_cloud);

        float theta = M_PI;

        Eigen::Affine3f transform_2 = Eigen::Affine3f::Identity();
        sensor_msgs::PointCloud2 rotated_cloud;

        pcl::PointCloud<pcl::PointXYZ>::Ptr transformed_cloud = rotatePC(source_cloud, transform_2, theta);

        //pcl::toROSMsg(transformed_cloud, rotated_cloud);
        pcl::toPCLPointCloud2(*transformed_cloud, pcl_pc2);
        pcl_pc2.header.frame_id = "point_cloud";


        pcl_pub.publish(pcl_pc2);
    }

protected:
    ros::NodeHandle nh;
    ros::Subscriber pcl_sub;
    ros::Publisher pcl_pub;
};
```

where I wrote the code in C++ to create a class called cloud_handler which would subscribe to the topic called “pcl_output”, where another node would publish first half of the point cloud and then my code rotates it 180 degrees and publish it to a topic called “pcl_rotated” which would then be subscribed by another topic created by my fellow intern and then scanning of the other side of the auger is done and both the scanned clouds (Rotated and Not rotated will be merged to get the final complete point-cloud.

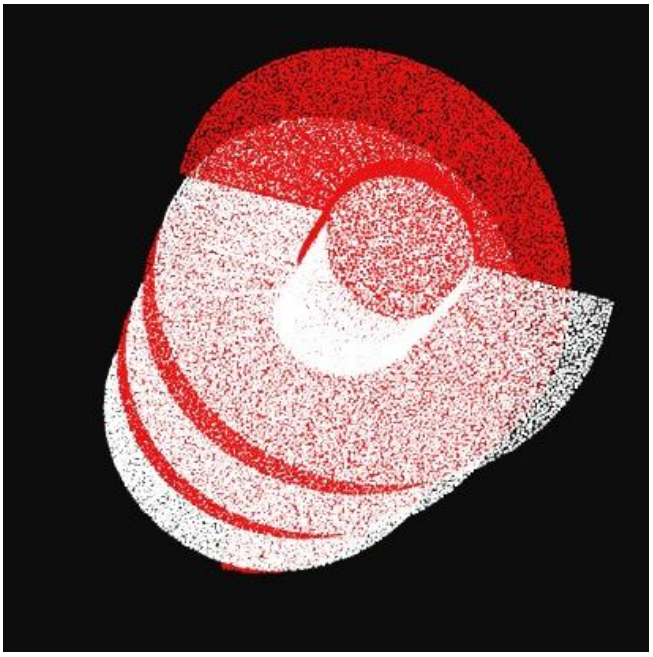


Fig 2.2.1 Rotated (Red) and Normal (White) Point Cloud

**2.3 Planning and Devising the Math required for simultaneous rotation of auger and horizontal movement of End-Effector of the Robotic Arm.**

This process involved me making some mistakes along the process and also learning along with it.

My fellow intern’s work got over with providing me the intersection points of the auger and the cylinder with the help of image processing techniques.

First approach that I followed was to extract the linear consecutive points by projecting all

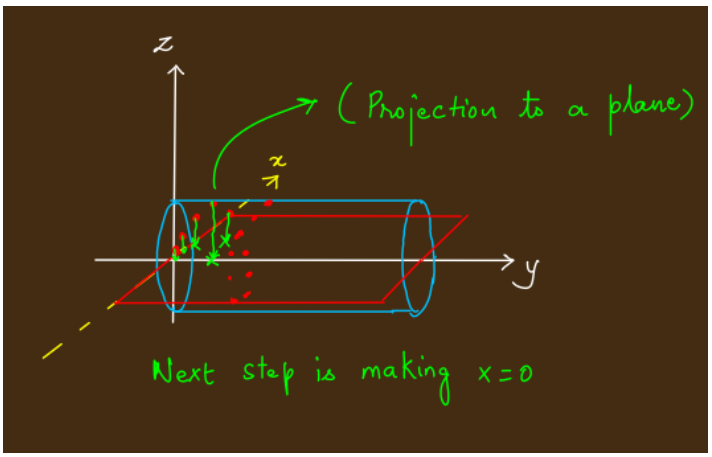



Fig 2.3.1

the points of intersection of auger and the cylinder (will be called seam here-after), to the axis parallel to the axis of the cylinder as shown in the Fig 2.3.1

But when I plotted the points, the method seemed to not work, and it wasn't giving the points that the end-effector would follow horizontally while welding the seam.

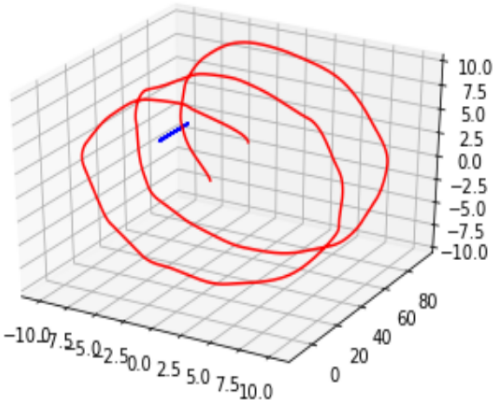
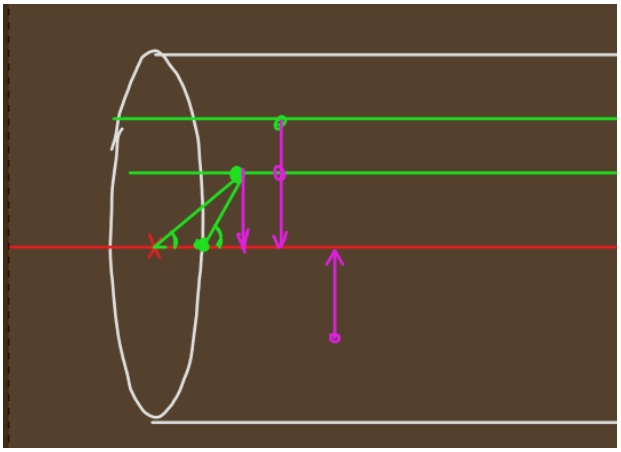



Fig 2.3.2 Plotting the calculated points

Also the first approach of calculating the angles was also a mistake as, I didn't calculate them w.r.t to the axis of the cylinder but instead, used the line as the base as shown in the figure 2.3.3, which is actually the horizontal line that lies on the surface of the cylinder.



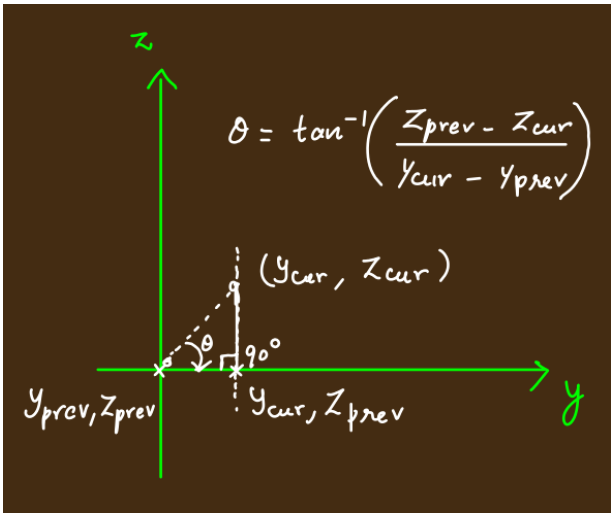



Fig 2.3.3

So these mistakes needed to be corrected, the next step that I focused on was just to figure out how to calculate the angles of rotation of the auger.

So, I created a node structure for calculation of angles, the structure looks like this.

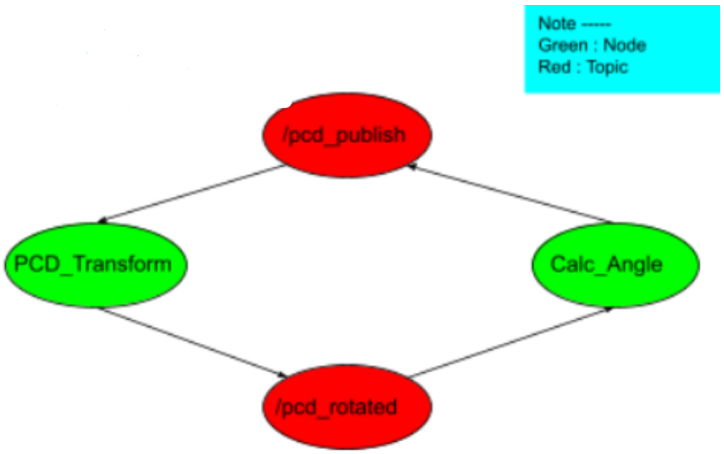



Fig 2.3.4

**PCD_Transform** Node will subscribe to the (/publish_pcd) topic and calculate the initial values of point cloud array and theta, and rotate it and publish the new point cloud array and theta to (/pcd_rotated). [In Loop]
**Calculate_Angle** Node will subscribe to the (/pcd_rotated) topic get the rotated point cloud array and theta, then, It will calculate the next angle, Save it to a file and then along with point cloud and new angle publish the message to (/pcd_publish) topic.

Calculating the angle is as follows. Referring to the Fig 2.3.5

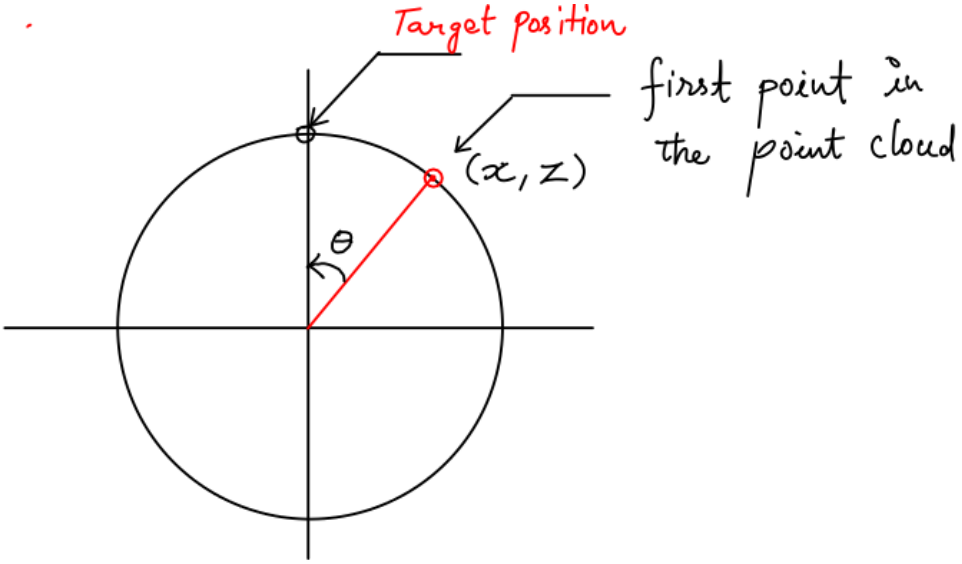



Fig 2.3.5

As you can see the angle is calculated (which is called axis angle as the angle being calculated here is w.r.t to the axis between the target point and the point before that in the seam numpy array.

These points are lying in the surface of the cylinder and Open-3D was used to rotate the point-cloud and generate the new set of points as a numpy array. But this process of calculating the angles using two ROS nodes didn't work, **though the method of calculating the angle is correct** because of ROS not being Robust in publishing and subscribing the calculated angles to the topics.

Later after discussion, I decided to calculate the angles offline, with a python code and then also calculate the positions of the points and store it to a numpy array which the arm has to follow horizontally in-order to weld the helix part to the cylinder.

**2.4 Algorithm for the above process**

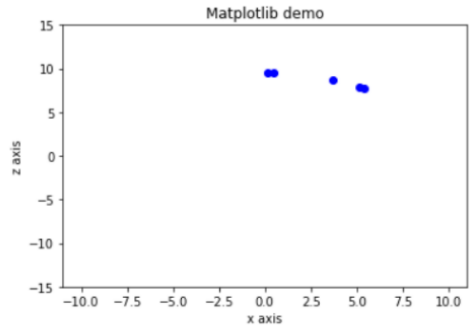
```
angles = []
#Final Rotation Code

red_pcd = o3d.io.read_point_cloud("/content/traj.pcd")

for i in range(len(red_pcd)):
 red_arr1 = PCDToNumpy(red_pcd)
 angle = np.arctan2(red_arr1[i][0], red_arr1[i][2])
 angles.append(angle)

 red_pcd_rotmat = copy.deepcopy(red_pcd)
 R =
red_pcd_rotmat.get_rotation_matrix_from_axis_angle(np.array([0
, -angle, 0]))

 red_pcd.rotate(R)
```

As you can see in the above code, first the point cloud of the seam is read and stored in the red_pcd variable, then in each run of the loop, it is converted to a numpy array, then angle is calculated as per the method stated in the previous section.

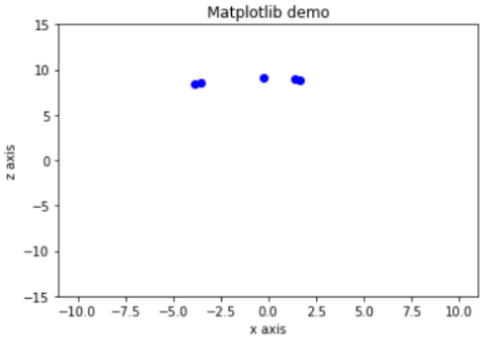
All angles are stored in a numpy array which will be used while welding later using ROS. In the next step, a deep-copy of red_pcd is created and that is used to get the rotation matrix for rotating the original red_pcd point cloud using Open-3D library's function, "get_rotation_matrix_from_axis_angle". As one can see from the last line of the code, R is the rotation matrix from the previous step which is used to rotate red_pcd and again rotate is also an inbuilt library function of Open-3D library.

Note: angle is passed as "-angle" to the code (Highlighted Part), because we need the points to rotate and assemble at the top surface of the cylinder in reference to the robotic arm for easy maneuvering. So using "-angle" perfectly does it for us as shown in the Fig 2.3.6

Sample points plotted




Snapshot of 2nd iteration




Snapshot of 3rd iteration

Fig 2.3.6

## 2.5 Rospy node for welding

```
def RotatePointCloud(self):

    r = rospy.Rate(0.5)

    print(self.move_group.get_current_rpy())

    with open('/home/ghans/catkin2_ws/src/kuka_arm_test/scripts/Correct_Angles.npy', 'rb') as f:
            angles = np.load(f)

    int_pcd = o3d.io.read_point_cloud("/home/ghans/catkin2_ws/src/kuka_arm_test/scripts/red2.pcd")

    for i in range(83):

            arr = np.asarray(int_pcd.points)

            r_header = std_msgs.msg.Header()
            r_header.stamp = rospy.Time.now()
            r_header.frame_id = self.planning_frame

            ros_pcd = pcl2.create_cloud_xyz32(r_header, arr.tolist())

            self.pub.publish(ros_pcd)

            self.go_to_pose_goal(arr, i)

            pcd_rotmat = copy.deepcopy(int_pcd)
            R = pcd_rotmat.get_rotation_matrix_from_axis_angle(np.array([0, -(angles[i]), 0]))

            int_pcd.rotate(R)

            r.sleep()
```

The function Rotate_point_cloud is written to perform rotation of the auger while welding, it takes the set of angles from the numpy file and in each run of the loop uses it to rotate the point-cloud.

To execute the welding process and to move from one point to another during subsequent runs of the loop, a function "go_to_pose_goal" is called which takes the current point cloud as an argument along with the point (index) it has to reach at every run of the loop.

```
    def go_to_pose_goal(self, arr, i):

        orient = quaternion_from_euler(-90.00000324031861 * (np.pi/180.0) ,
1.272221883397151e-14 * (np.pi/180.0) , -90.00000324031861 * (np.pi/180.0))

        pose_goal = geometry_msgs.msg.Pose()

        pose_goal.orientation = geometry_msgs.msg.Quaternion(*orient)

        pose_goal.position.x = arr[i][0]
        pose_goal.position.y = arr[i][1]
        pose_goal.position.z = arr[i][2]

        self.move_group.set_pose_target(pose_goal)

        plan = self.move_group.go(wait=True)

        self.move_group.stop()

        self.move_group.clear_pose_targets()

        current_pose = self.move_group.get_current_pose().pose
        return all_close(pose_goal, current_pose, 0.01)
```

This go_to_pose_goal function uses moveit's default cartesian path planner, which takes the orientation of the arm after reaching the point and the three cartesian coordinates that it has to reach at every run of the loop and calculates the path, which is later executed in gazebo.

# Results and Conclusion

**https://drive.google.com/file/d/1sYNmZ-oXY0lgNGADIOTX_Yb5ocNVgvPg/view?resourcekey** This is the link of the final execution of the algorithm I developed to perform welding. By this I conclude that it is possible to weld an auger autonomously with minimum human intervention as proposed in the aim of the project, at-least in simulation for now, which completes the feasibility study of this research project. Finally this can be extended and modified in-order to implement it physically with a real robotic manipulator with similar conditions as done above.

# References


- "Move Group Interface Python Tutorial." https://ros-planning.github.io/moveit_tutorials/doc/move_group_python_interface/move_group_python_interface_tutorial.html
- From Wikipedia, the free encyclopedia "Axis-Angle Representation" https://en.wikipedia.org/wiki/Axis%E2%80%93angle_representation
- "Quaternions" http://wiki.ros.org/tf2/Tutorials/Quaternions
- "Open-3D geometry" http://www.open3d.org/docs/latest/python_api/open3d.geometry.get_rotation_matrix_from_axis_angle.html
- "Using a matrix to transform a point cloud" https://pointclouds.org/documentation/tutorials/matrix_transform.html